

Experiment No.	1
Experiment Title	Working with Python Packages (NumPy, Pandas, SciPy, Scikit Learn and Matplotlib) 1
Student Name	Harshita S
Register Number	3122247001020
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27/07/2026

1 Objective

To explore the functionalities of Python libraries including NumPy, Pandas, SciPy, Scikit-Learn, and Matplotlib by implementing a complete machine learning workflow using the Iris dataset. The experiment includes dataset loading, exploratory data analysis, preprocessing, feature selection, classification using multiple machine learning algorithms, and performance evaluation.

2 Problem Statement

The objective of this experiment is to understand the complete machine learning workflow using the Iris dataset. The dataset is explored through statistical analysis and visualization, preprocessed using encoding and feature scaling techniques, followed by feature selection. Multiple classification algorithms such as Random Forest, Decision Tree, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM) are trained and evaluated using standard classification metrics to compare their performance.

3 Dataset Description

Dataset Name	Iris Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	150
Number of Features	4
Number of Classes	3
Missing Values	0
Duplicate Samples	3
Train-Test Split	80% Training, 20% Testing
Feature Selection Method	ANOVA (SelectKBest)
Target Variable	Species

4 Exploratory Data Analysis

The Iris dataset was explored using a reusable Exploratory Data Analysis (EDA) function developed in Python. A consolidated EDA figure containing 12 subplots was generated to provide a comprehensive understanding of the dataset. The visualizations include dataset information, statistical summary, missing value analysis, feature distributions, boxplots,

scatter plots, class distribution, and correlation analysis. The generated EDA summary figure was included in the report to facilitate visualization and analysis of the dataset.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

EDA Summary Figure

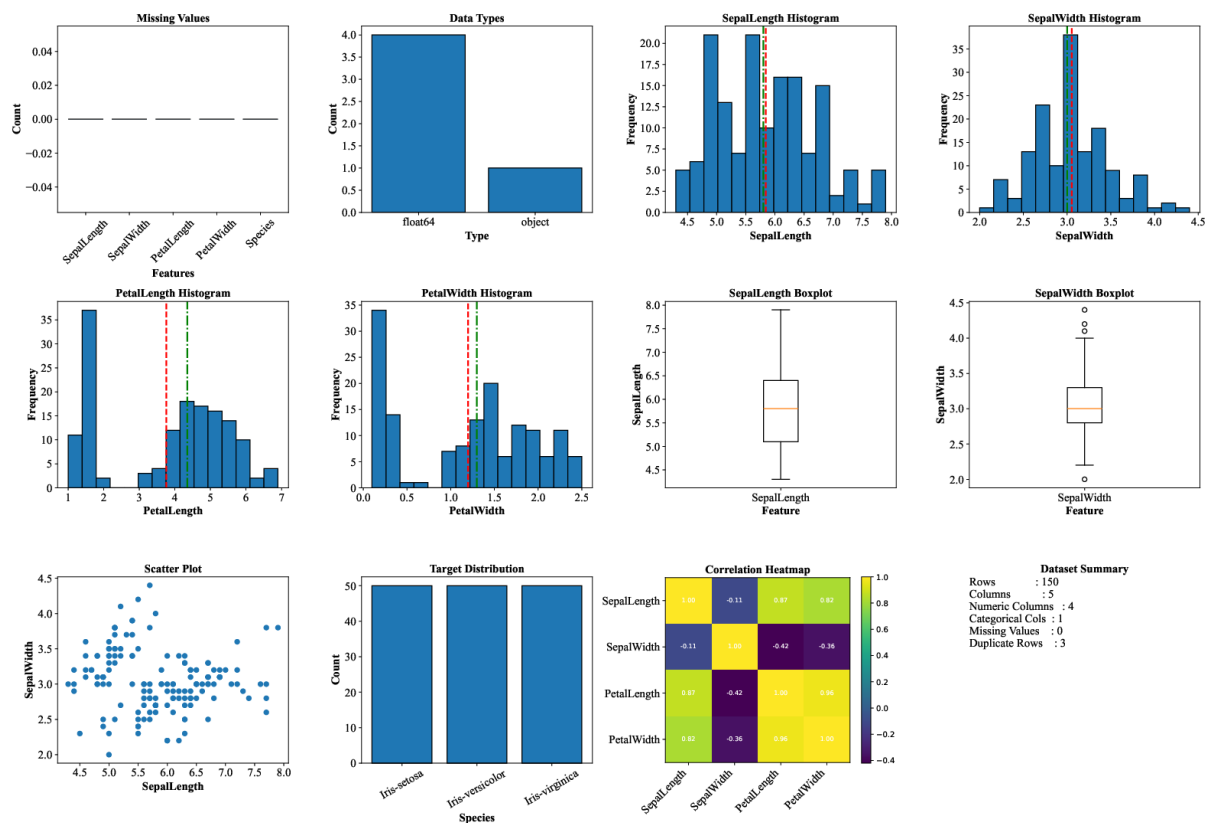


Figure 1: Comprehensive Exploratory Data Analysis (EDA) of the Iris Dataset

Inference:

The consolidated EDA figure provides a comprehensive overview of the Iris dataset. The dataset contains three balanced flower species with no missing values and only a few duplicate records. Petal length and petal width exhibit stronger discrimination among the species than the sepal measurements. The correlation heatmap shows a strong positive relationship between petal length and petal width, indicating that these features are highly informative for classification. Overall, the dataset is clean, well-balanced, and suitable for supervised machine learning.

5 Data Preprocessing

The raw Iris dataset was prepared for machine learning using a series of preprocessing steps.

- **Missing Value Handling:**

The dataset was checked for missing values. No missing values were found, therefore no imputation technique was required.

- **Encoding:**

The target variable “Species” contains categorical class labels. Label encoding was applied to convert the categorical species names into numerical class values suitable for machine learning algorithms.

- **Feature Scaling:**

Standardization was performed using StandardScaler. The numerical features were transformed to have zero mean and unit variance. Feature scaling improves the performance of distance-based algorithms such as KNN and SVM.

- **Train-Test Split:**

The dataset was divided into training and testing sets using an 80:20 ratio. Stratified splitting was applied to maintain equal class distribution in both training and testing datasets.

- **Feature Selection:**

ANOVA based SelectKBest feature selection was applied. Since the Iris dataset contains only four meaningful features, all features were retained by setting $k = all$.

6 Machine Learning Algorithms

Four supervised classification algorithms were implemented and compared.

6.1 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy. Each tree is trained using randomly selected samples and features, and the final prediction is obtained through majority voting. **Mathematical Formulation:**

Suppose there are N decision trees trained on different bootstrap samples of the dataset. The final prediction of the Random Forest classifier is obtained by majority voting:

$$\hat{y} = \text{mode} \{T_1(x), T_2(x), \dots, T_N(x)\}$$

where $T_i(x)$ represents the prediction made by the i^{th} decision tree.

Advantages

- Handles complex datasets effectively.

- Reduces overfitting compared to individual decision trees.
- Works well with numerical and categorical features.

Limitations:

- Requires higher computational resources.
- Less interpretable compared to a single decision tree.

6.2 Decision Tree

Decision Tree classification uses a tree structure where data is split based on feature values to maximize class separation.

Mathematical Formulation:

Decision Trees split the data by maximizing information gain based on entropy.

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where p_i is the probability of class i .

Advantages

- Easy to interpret.
- Requires minimal preprocessing.

Limitations:

- Can overfit small datasets.
- Sensitive to small changes in data.

6.3 K-Nearest Neighbors (KNN)

KNN classifies a sample based on the majority class among its nearest neighboring samples.

The distance between samples is calculated using:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Advantages

- Simple implementation.
- Effective for small datasets.

Limitations:

- Prediction time increases with dataset size.
- Sensitive to feature scaling.

6.4 Support Vector Machine (SVM)

SVM finds an optimal hyperplane that separates different classes by maximizing the margin between support vectors.

Mathematical Formulation:

The separating hyperplane is represented as

$$w^T x + b = 0$$

where

- w is the weight vector.
- b is the bias term.

The objective is to maximize the margin between the support vectors of different classes.

Advantages

- Effective in high dimensional spaces.
- Provides good generalization.

Limitations:

- Parameter tuning is required.
- Less suitable for very large datasets.

7 Experimental Setup

The experiment was implemented using Python in a Jupyter Notebook environment. The complete machine learning workflow was developed using reusable Python modules and executed using the prescribed Anaconda environment.

Component	Specification
Python Version	Python 3.12
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	Ubuntu/Linux (Anaconda Environment)
Processor	Intel(R) Core(TM) i7-10610U CPU @ 1.80 GHz
Memory	16 GB RAM
System Type	64-bit Operating System, x64-based Processor

Table 2: Experimental Setup

8 Hyperparameter Selection

The default hyperparameters provided by Scikit-Learn were used for the classification models.

Parameter	Value
Random Forest (n_estimators)	100
Random Forest (random_state)	42
Decision Tree (criterion)	Gini
Decision Tree (random_state)	42
KNN (n_neighbors)	5
SVM (kernel)	RBF
Feature Selection	ANOVA (SelectKBest)
Selected Features (k)	all

9 Results

The classification performance of the implemented machine learning algorithms was evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC. The results are summarized in Table 1.

Table 3: Performance Comparison of Classification Models

Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC
Random Forest	0.9667	0.9697	0.9667	0.9666	0.9900
Decision Tree	0.9333	0.9333	0.9333	0.9333	0.9500
KNN	0.9333	0.9444	0.9333	0.9327	0.9933
SVM	0.9667	0.9697	0.9667	0.9666	0.9967

Inference:

The experimental results show that Random Forest and Support Vector Machine achieved the highest classification accuracy of 96.67%. SVM obtained the highest ROC-AUC score of 0.9967, indicating excellent class separation capability. Decision Tree and KNN achieved slightly lower accuracy of 93.33%. Overall, ensemble and margin-based classifiers performed better for the Iris dataset.

10 Result Visualization

The classification models were further evaluated using confusion matrices, which provide a detailed comparison between the actual and predicted class labels. These visualizations help identify correctly classified samples as well as any misclassifications made by each classifier.

10.1 Random Forest Classifier

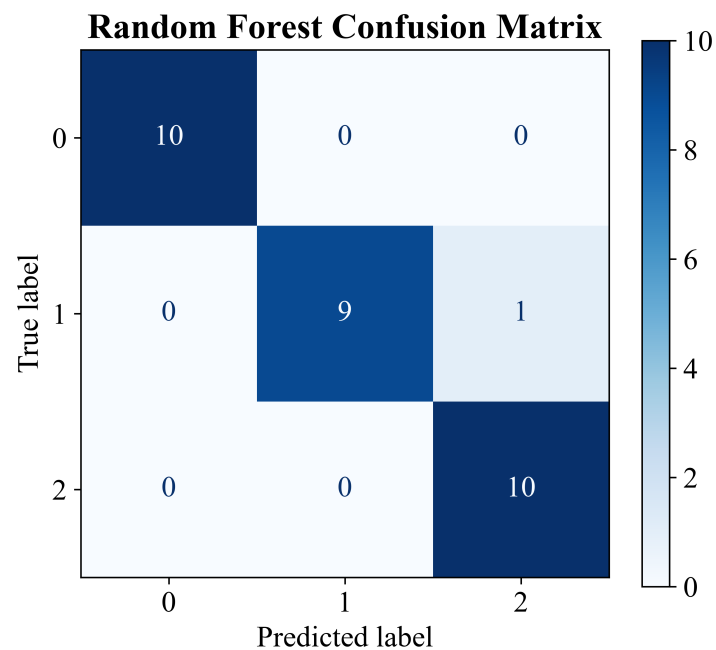


Figure 2: Confusion Matrix of the Random Forest Classifier

Inference:

The Random Forest classifier correctly classified 29 out of the 30 testing samples, achieving an overall accuracy of 96.67%. Only one *Iris-versicolor* sample was misclassified as *Iris-virginica*. The confusion matrix indicates that the classifier effectively separates the three Iris species with minimal classification error.

10.2 Decision Tree Classifier

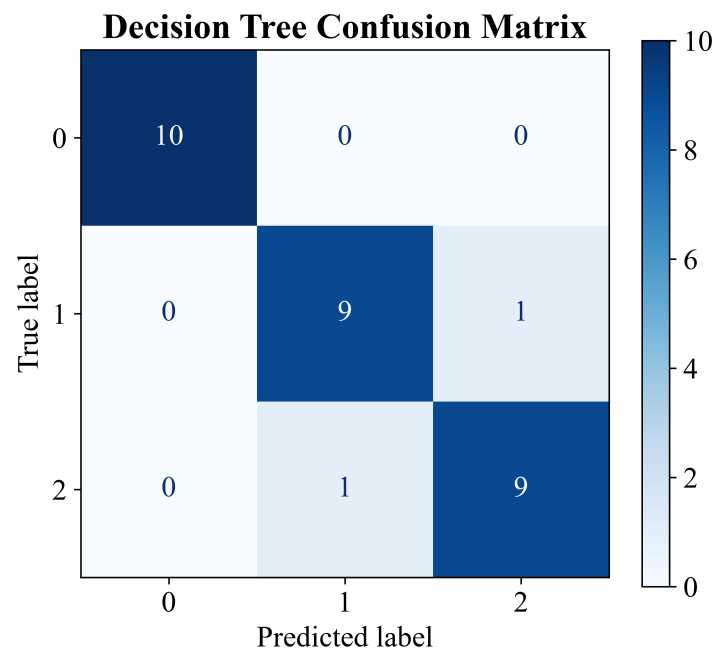


Figure 3: Confusion Matrix of the Decision Tree Classifier

Inference:

The Decision Tree classifier achieved an accuracy of 93.33%. Most samples were classified correctly; however, one *Iris-versicolor* sample and one *Iris-virginica* sample were misclassified. This suggests that these two species exhibit similar feature characteristics, making them comparatively more difficult to distinguish.

10.3 K-Nearest Neighbors (KNN)

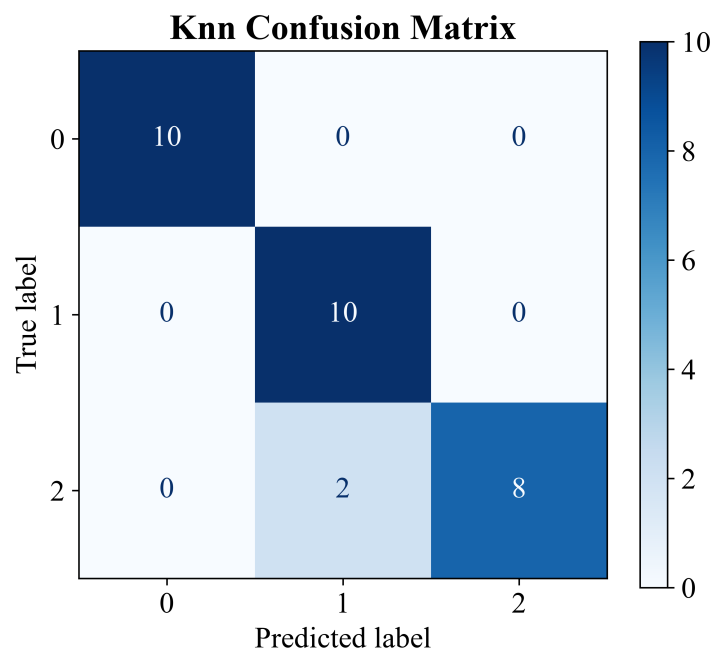


Figure 4: Confusion Matrix of the K-Nearest Neighbors Classifier

Inference:

The KNN classifier correctly identified all *Iris-setosa* and *Iris-versicolor* samples. However, two *Iris-virginica* samples were incorrectly classified as *Iris-versicolor*. Since KNN is a distance-based algorithm, the overlap between these two species in the feature space contributes to these misclassifications.

10.4 Support Vector Machine (SVM)

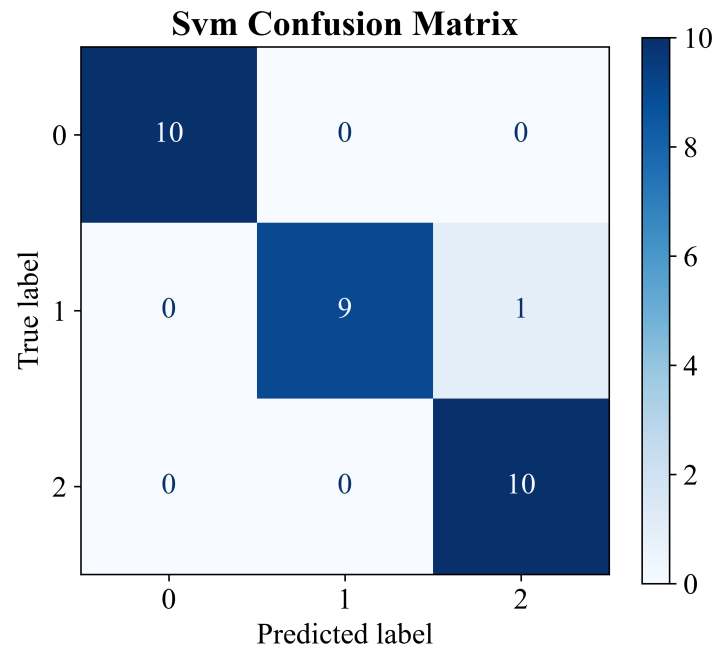


Figure 5: Confusion Matrix of the Support Vector Machine Classifier

Inference:

The Support Vector Machine classifier achieved an accuracy of 96.67%, correctly classifying 29 of the 30 testing samples. Similar to the Random Forest classifier, only one sample was misclassified. Combined with the highest ROC-AUC score of 0.9967, the confusion matrix demonstrates the excellent classification capability of the SVM model.

11 Discussion

This experiment successfully demonstrated the complete machine learning workflow using the Iris dataset, beginning with data loading and exploratory data analysis, followed by preprocessing, feature selection, model training, and performance evaluation. The exploratory analysis confirmed that the dataset is balanced, contains no missing values, and exhibits strong correlations between petal length and petal width, making these features highly informative for classification.

Standardization of the numerical features improved the performance of distance-based algorithms such as K-Nearest Neighbors and Support Vector Machine. ANOVA-based feature selection was also applied; however, all four features were retained because each contributed meaningful information to the classification task.

Among the evaluated models, Random Forest and Support Vector Machine achieved the highest classification accuracy of 96.67%. The Support Vector Machine also produced the highest ROC-AUC score (0.9967), indicating superior discriminative capability. Decision Tree and KNN achieved an accuracy of 93.33%, with only a few misclassifications occurring between the *Iris-versicolor* and *Iris-virginica* classes due to similarities in their feature distributions.

Overall, the experiment highlights the importance of proper preprocessing, feature selection, and model evaluation in building reliable machine learning models. Although the Iris dataset is relatively small and well-structured, the same workflow can be extended to larger and more complex datasets by incorporating techniques such as cross-validation, hyperparameter tuning, and ensemble learning.

12 Conclusion

This experiment provided practical experience in implementing a complete machine learning pipeline using Python libraries including NumPy, Pandas, Matplotlib, and Scikit-Learn. The workflow included dataset loading, exploratory data analysis, preprocessing, feature selection, model training, and performance evaluation using multiple supervised classification algorithms.

Four classification models—Random Forest, Decision Tree, K-Nearest Neighbors, and Support Vector Machine—were trained and compared on the Iris dataset. Random Forest and Support Vector Machine achieved the highest classification accuracy of 96.67%, while the Support Vector Machine produced the highest ROC-AUC score of 0.9967, demonstrating excellent classification performance.

The experiment reinforced the significance of exploratory data analysis, preprocessing, appropriate feature selection, and quantitative performance evaluation in developing accurate and reliable machine learning models. The reusable implementation developed during this experiment also provides a scalable framework that can be applied to future machine learning tasks.

13 References

1. Scikit-learn Developers, *Scikit-learn Documentation*. Available: <https://scikit-learn.org/stable/>
2. Dua, D., and Graff, C., *UCI Machine Learning Repository: Iris Data Set*. Available: <https://archive.ics.uci.edu/ml/datasets/iris>
3. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed., O'Reilly Media, 2023.